



Confidential material: This page is confidential. Do not share or discuss until authorized to do so.

[Gemini Deep Research \(/gemini-api/docs/deep-research\)](/gemini-api/docs/deep-research) is now available in preview with collaborative planning, visualization, MCP support, and more.

Gemini Agents API

The Gemini Agents API lets you build managed, autonomous agents with a single API call. Powered by the Waverunner harness, each agent runs in a secure, sandboxed environment where it can reason, plan, execute code, search the web, and read and write files — all without human intervention.

With the Agents API, you can:

- **Go from prompt to autonomous agent:** With a single POST request to the `/interactions` endpoint, spin up an agent that handles the full loop — planning, tool use, verification — so you can focus on the user's goal rather than orchestration.
- **Persist state across turns:** Environments are managed Linux sandboxes that persist files across interactions. Reuse an `environment_id` to continue where you left off, enabling multi-turn, stateful workflows.
- **Create reusable agents:** Bundle instructions, tools, and environment configuration into a named agent via the `/agents` endpoint, then invoke it by ID from any client or schedule it with triggers.
- **Define agents as files:** An agent is just a set of configurations, an `AGENTS.md` file, and optional skills — which you can bring from either your local environment or different cloud sources such as GitHub and GCS.

Your First Agent Call

To interact with the default agent, Waverunner, you send a POST request to the `/interactions` endpoint.

```
curl -X POST "https://generativelanguage.googleapis.com/v1beta/interactions" \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $GEMINI_API_KEY" \
-d '{
  "agent": "waverunner",
  "input": [{"type": "text", "text": "What is 2 + 2?"}],
  "environment": {"enabled": true},
  "stream": false
}'
```

This request returns a complete response as a single JSON object, including the final answer and a summary of work:

```
{
  "id": "v1_...",
  "status": "completed",
  "outputs": [
    {
      "text": "2 + 2 is 4.\n\n### Summary of work",
      "type": "text"
    },
    {
      "text": "\n- Answered the user's arithmetic question.",
      "type": "text"
    }
  ],
  "environment_id": "...",
  "object": "interaction",
  "agent": "waverunner"
}
```

And with that you've made your first interaction with the Waverunner agent. This single API call provisioned a dedicated sandbox for Waverunner to operate in.

Notice the `"environment_id"` in the response above. You can persist this sandbox environment across multiple API calls to maintain state. To do this, pass this ID as `env_id` in your subsequent requests.

```
curl -X POST "https://generativelanguage.googleapis.com/v1beta/interactions" \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $GEMINI_API_KEY" \
```

```
-d '{
  "agent": "waverunner",
  "input": [{"type": "text", "text": "What was the answer to my previous question?"}
  "environment": {"env_id": "YOUR_ENVIRONMENT_ID"},
  "previous_interaction_id": "YOUR_PREVIOUS_INTERACTION_ID",
  "stream": false
}'
```

To continue a multi-turn conversation, you also need to link your new request to a previous one. Extract the `id` from a previous turn's completion event and pass it as `previous_interaction_id` in your next request to maintain conversation context.

Creating a Custom Agent

The recommended flow is to start building quickly by using the base `waverunner` agent. Once you have iterated on the instructions and environment in your interactions, you can persist that state by creating a named custom agent. Just use the environment ID from your successful interaction to fork it into a reusable agent definition.

First, create an interaction and specify the sources you want to load into the environment. Here we load custom skills for creating slides from a Google Cloud Storage bucket.

```
curl -X POST "https://generativelanguage.googleapis.com/v1beta/interactions" \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $GEMINI_API_KEY" \
-d '{
  "agent": "waverunner",
  "input": [{"type": "text", "text": "List the skills available."}],
  "environment": {
    "config": {
      "sources": [
        {"type": "GCS", "source": "gs://eap-templates/slides-skill", "target": ""}
      ]
    }
  },
  "stream": false
}'
```

Now you can use that `environmentId` to create a named agent using the `/agents` endpoint. The `base_environment` field accepts either an `env_id` to fork from an existing environment, or a `config`

with sources to build the environment from scratch.

Option 1: Fork from an existing environment

```
curl -X POST "https://generativelanguage.googleapis.com/v1beta/agents" \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $GEMINI_API_KEY" \
-d '{
  "name": "my-custom-slides-agent",
  "base_agent": "waverunner",
  "instructions": "You are a helpful assistant that creates slides.",
  "base_environment": {"env_id": "YOUR_ENVIRONMENT_ID"}
}'
```

Option 2: Define sources directly

```
curl -X POST "https://generativelanguage.googleapis.com/v1beta/agents" \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $GEMINI_API_KEY" \
-d '{
  "name": "my-custom-slides-agent",
  "base_agent": "waverunner",
  "instructions": "You are a helpful assistant that creates slides.",
  "base_environment": {
    "config": {
      "sources": [
        {"type": "GCS", "source": "gs://eap-templates/slides-skill", "target": ""}
      ]
    }
  }
}'
```

Invoking your Custom Agent

Once you have created a named agent, you can interact with it by specifying its resource name in the `agent` field of the `/interactions` request, instead of the default `waverunner`.

```
curl -X POST "https://generativelanguage.googleapis.com/v1beta/interactions" \
-H "Content-Type: application/json" \
```

```
-H "x-goog-api-key: $GEMINI_API_KEY" \  
-d '{  
  "agent": "my-custom-slides-agent",  
  "input": [{"type": "text", "text": "Create a 5-slide presentation about AI trends"}],  
  "environment": {"enabled": true},  
  "stream": false  
'
```

Manage agents

The `/agents` endpoint supports full CRUD operations for managing your custom agents.

```
# List all agents  
curl "https://generativelanguage.googleapis.com/v1beta/agents" \  
  -H "x-goog-api-key: $GEMINI_API_KEY"  
  
# Get a specific agent  
curl "https://generativelanguage.googleapis.com/v1beta/agents/my-custom-slides-agent" \  
  -H "x-goog-api-key: $GEMINI_API_KEY"  
  
# Delete an agent  
curl -X DELETE "https://generativelanguage.googleapis.com/v1beta/agents/my-custom-slides-agent" \  
  -H "x-goog-api-key: $GEMINI_API_KEY"  
  
# List versions of an agent  
curl "https://generativelanguage.googleapis.com/v1beta/agents/my-custom-slides-agent/versions" \  
  -H "x-goog-api-key: $GEMINI_API_KEY"
```

Now let's look at what's happening behind the scenes in terms of capabilities.

Environment Details

The agent operates within a secure, isolated Linux container. When you customize the environment with sources (like GitHub or GCS), these files are placed in the container's filesystem.

Any modifications the agent makes to the filesystem during an interaction (such as creating new files or modifying existing ones) are preserved within that specific environment. By capturing the `environment_id` from a response, you can continue working in that exact same state in subsequent calls.

You can customize the agent's sandbox by injecting files before execution starts.

The API supports three sources for these files: Google Cloud Storage, GitHub repositories, and direct inline content.

Environment field options

The `environment` field in an interaction request supports three mutually exclusive options:

Option	Description	Use case
<code>{"enabled": true}</code>	Provision a new, clean environment.	First interaction or one-off tasks.
<code>{"env_id": "..."} </code>	Reuse an existing environment by ID.	Multi-turn workflows with persisted state.
<code>{"config": {"sources": [...]} }</code>	Provision a new environment and preload files from GCS, GitHub, or inline content.	First interaction with custom skills or data.

You can combine `env_id` with `config` to reuse an existing environment and load additional sources into it.

Pre-installed software

The execution environment is based on **Debian** and includes the following pre-installed software:

- **UNIX Tools:** `curl`, `wget`, `git`, `rsync`, `unzip`, `ripgrep`, `fd-find`, `gawk`, `bc`, `tree`, `which`, `lsof`, `htop`, `jq`, `iproute2`, `procs`, `gcloud` CLI
- **Python 3.11 Packages:** `numpy`, `pandas`, `requests`, `google-genai`, `beautifulsoup4`, `pyyaml`, `ast-grep-cli`
- **Node.js 20 Global Packages:** `create-next-app`, `create-vite`, `typescript`

The agent can install additional packages at runtime using `pip install` or `npm install`. Any packages installed during an interaction are persisted in the environment snapshot and available in subsequent interactions that reuse the same `environment_id`.

Here is an example showing how to configure all three source types in a single request.

```
curl -X POST "https://generativelanguage.googleapis.com/v1beta/interactions" \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $GEMINI_API_KEY" \
-d '{
  "agent": "waverunner",
  "input": [{"type": "text", "text": "List files in the sandbox"}],
  "environment": {
    "config": {
      "sources": [
        {"type": "INLINE", "content": "your content", "target": "/credentials"},
        {"type": "GCS", "source": "gs://my-bucket/data/", "target": "/"},
        {"type": "GITHUB", "source": "https://github.com/user/repo.git", "tar
      ]
    }
  },
  "stream": true
}'
```

Download environment snapshots

You can download the files that were created or modified during an interaction as a tar file. Use the `files/environment-{environment_id}` resource with the Files API download endpoint.

```
ENVIRONMENT_ID="YOUR_ENVIRONMENT_ID"
```

```
curl -L -o snapshot.tar \
  "https://generativelanguage.googleapis.com/v1beta/files/environment-${ENVIRONMENT_ID}" \
-H "x-goog-api-key: $GEMINI_API_KEY"
```

```
# Unpack the snapshot
mkdir -p extracted_snapshot
tar xf snapshot.tar -C extracted_snapshot
ls extracted_snapshot
```

Note: The `-L` flag is required because the download endpoint returns a redirect to the file content.

Agent Capabilities

The agent operates in a secure sandbox where it can perform several actions. It can write and run code, and browse the web for information. The agent can also read and write files within this sandbox environment. However, it is currently constrained to reading text files only. When running in the managed environment, the agent has access to several default tools. These tools are available to the model via function calling.

Tool Name	Description
bash	Executes a shell command and returns the output.
url_context	Retrieves and extracts text content from specified URLs.
google_search	Performs a web search to provide results.
delete_file	Removes a file from the filesystem.
list_files	Lists the contents of a directory.
read_file	Reads and returns the contents of a file.
write_file	Creates or overwrites a file with specified content.

Streaming Interactions

For tasks that take longer to execute, you can enable streaming by setting `stream: true`. This allows you to receive real-time updates as the agent works, rather than waiting for the final complete response.

The stream delivers a series of events, which can include partial text generation as it happens, as well as notifications when the agent decides to call a tool and the results of that tool execution.

```
curl -X POST "https://generativelanguage.googleapis.com/v1beta/interactions" \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $GEMINI_API_KEY" \
-d '{
  "agent": "waverunner",
```



```
"input": [{"type": "text", "text": "Run a complex analysis on the codebase."}],  
"environment": {"enabled": true},  
"stream": true  
'
```

The API returns a Server-Sent Events (SSE) stream. Each `data:` line is a JSON object, except for the final line which signals completion with a literal `[DONE]` string (not JSON). You should check for this to stop processing.

```
if (dataStr === "[DONE]") return;
```

The structure of `content.delta` varies depending on the type of content being streamed:

Content Type	Delta Path	Description
Thought	<code>event.delta.content.text</code>	The agent's internal reasoning.
Text	<code>event.delta.text</code>	The final output for the user.
Function Call	<code>event.delta</code>	Contains name and arguments (check type).
Function Result	<code>event.delta</code>	Contains the result of a tool call (check type).

Building agents with the Waverunner CLI

Once you are ready to move from simple API calls to full agent development, you can define your custom agent entirely through files. An agent directory contains configuration, system instructions, optional skills, and workspace data. You can create, iterate on, and deploy an agent by manipulating these files, either manually or with the help of a coding agent.

Install the Waverunner CLI

The `waverunner` CLI makes it easy for both humans and AI to build, test, and deploy agents.

To install the CLI binary:

```
gcloud storage cp gs://gemini-api-eap/agents-api/install.sh ./tmp/install.sh
bash ./tmp/install.sh
```

Install the Skill for your AI Agent

If you are using an AI Coding Agent (like Antigravity), you should install the `waverunner-cli` skill so it knows how to use the tool:

```
mkdir -p tmp/skills/waverunner-cli
gcloud storage cp gs://gemini-api-eap/agents-api/skills/waverunner-cli/SKILL.md tmp/s
npx skills add ./tmp/skills/waverunner-cli
```

Agent directory structure

A standard agent directory looks like this:

```
my-agent/
├── agent.yaml      # Configuration (model, tools, environment)
├── AGENTS.md       # System instructions (loaded automatically)
├── skills/         # Custom skills (SKILL.md files)
└── workspace/     # Files seeded into the remote environment
```

- **agent.yaml**: Defines the agent's configuration — its base agent, tools, environment sources, and metadata. This maps directly to the `/v1/agents` API resource.
- **AGENTS.md**: Contains system instructions for the agent. If present in `.agents/` or the workspace root, it is automatically loaded.
- **skills/**: Optional directory containing **SKILL.md** files that extend the agent's capabilities. Skills are automatically registered by the harness.
- **workspace/**: Files placed here are seeded into the remote environment at startup.

agent.yaml

The `agent.yaml` file requires at minimum an `id` and `base_agent`. Currently the only supported base agent is `waverunner`.

```
id: my-agent
base_agent: waverunner
description: "My agent description"
system_instruction: "You are a helpful assistant."
tools:
  - type: code_execution
  - type: google_search
environment:
  enabled: true
```

Example Prompt for your AI Agent

You can ask your AI agent to create a specific agent for you. For example:

"Create a new Gemini agent called 'data-expert' that can analyze CSV files. It should have access to code execution and google search. Seed the workspace with a sample CSV file and add instructions on how to analyze it."

What the AI Agent Will Do

When you give it a prompt like the one above, the AI agent will automatically perform the following steps:

1. **Scaffold:** Run `waverunner init data-expert` to create the project structure. You can also use `--from-template <url>` to scaffold from an existing template.
2. **Configure:** Edit `agent.yaml` to set the correct model and enable tools (`code_execution`, `google_search`).
3. **Instruct:** Write instructions in `AGENTS.md` detailing how the agent should behave and analyze data.
4. **Seed Workspace:** Create the sample CSV file in the `workspace/` directory.
5. **Local Test:** Run `waverunner test --prompt "Analyze the sample data"` to verify the agent works as expected locally.
6. **Deploy:** Once verified, run `waverunner create` to deploy the agent to the platform.

Limitations

- **Production use:** As an EAP feature, the API is subject to change and is not recommended for use in production applications.
- **Text files only:** The agent is currently constrained to reading text files. Binary file support is not yet available.
- **Inline source size:** Inline sources (`type: "INLINE"`) are limited to 75 KB per file. For larger files, use GCS or GitHub sources instead.
- **Tool combination:** It is not currently possible to combine custom function declarations with the agent's built-in tools in the same request.
- **Environment startup:** Environment provisioning typically takes up to 5 seconds. Large source repositories may increase this time.
- **Session timeout:** Environments have configurable timeout and inactivity thresholds. Long-running processes may be terminated automatically.

All rights reserved. Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-04-21 UTC.